PRÁCTICA: Agente Auditor de Tiempo

Objetivo: Evaluar la creación de herramientas de análisis sobre Calendar, usar SimpleMemory y orquestar 2 tools.

APIs:

- service.freebusy().query() -> para calcular horas ocupadas
- service.events().list(q="palabra clave") -> búsqueda inteligente por texto
- service.calendars().get() -> info del calendario principal

Enunciado para alumno:

Eres un asistente de productividad. El usuario no quiere que crees ni borres nada, solo que analices su agenda.

Debes crear 2 tools nuevas:

1. analizar_dia(time_ini, time_end):
Llama a freebusy + events.list y retorna JSON analítico:

{ "success": true, "total_reuniones": 5, "horas_ocupadas": 3.5, "primera_reunion": "09:00 - Daily", "ultima_reunion": "17:00 - Review", "huecos_libres": ["11:00-11:30", "14:00-15:00"]}

Debes calcular en Python las horas ocupadas sumando busy y encontrar huecos.

2. buscar_por_tema(time_ini, time_end, tema):
Usa events().list(calendarId="primary", timeMin=..., timeMax=..., q=tema)
El parámetro q es búsqueda de texto libre de Google Calendar.

Retorna cuántas reuniones contienen ese tema y lista de títulos.
Ej: tema="cliente" -> busca todas las reuniones con cliente en título o descripción.

Flujo obligatorio con memoria:
Si el usuario dice: "¿Cuántas reuniones de cliente tuve ayer?" -> llamas a buscar_por_tema
Si luego dice "¿Y cuánto tiempo libre tuve?" -> debes usar el mismo time_ini/time_end de memoria, sin preguntar fecha de nuevo.

SYSTEM_PROMPT anti-alucinación: "NUNCA inventes número de reuniones. Siempre llama a analizar_dia o buscar_por_tema. Si no hay datos, di 'No se encontraron eventos' no inventes."

Rúbrica (10 pts):

- *40 pts: analizar_dia calcula bien horas ocupadas y huecos (lógica Python)*
- *30 pts: buscar_por_tema usa parámetro q correctamente*
- *20 pts: Uso de memoria para no volver a pedir fecha*
- *10 pt: SYSTEM_PROMPT anti-alucinación*